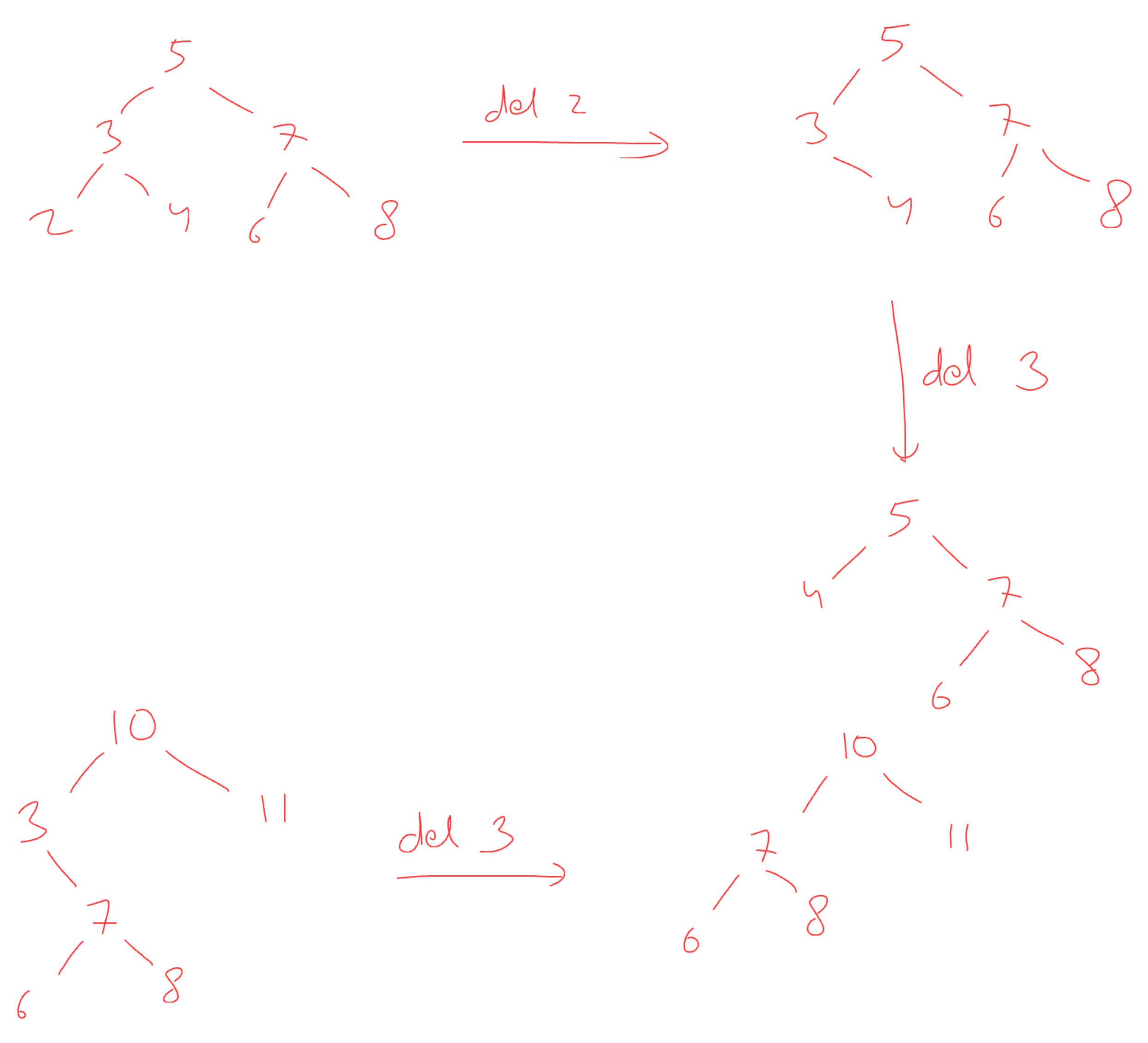

del 2
del 3
del 3

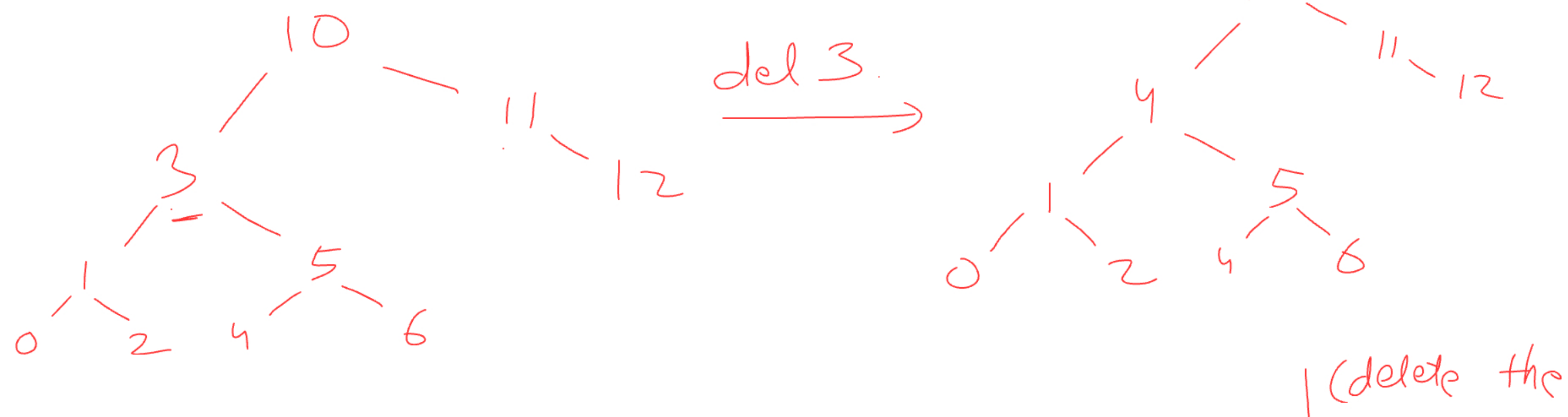
case-1 : 0 child nodes : Replace with NULL

case-2 : 1 child node : Replace with the non-NULL child node

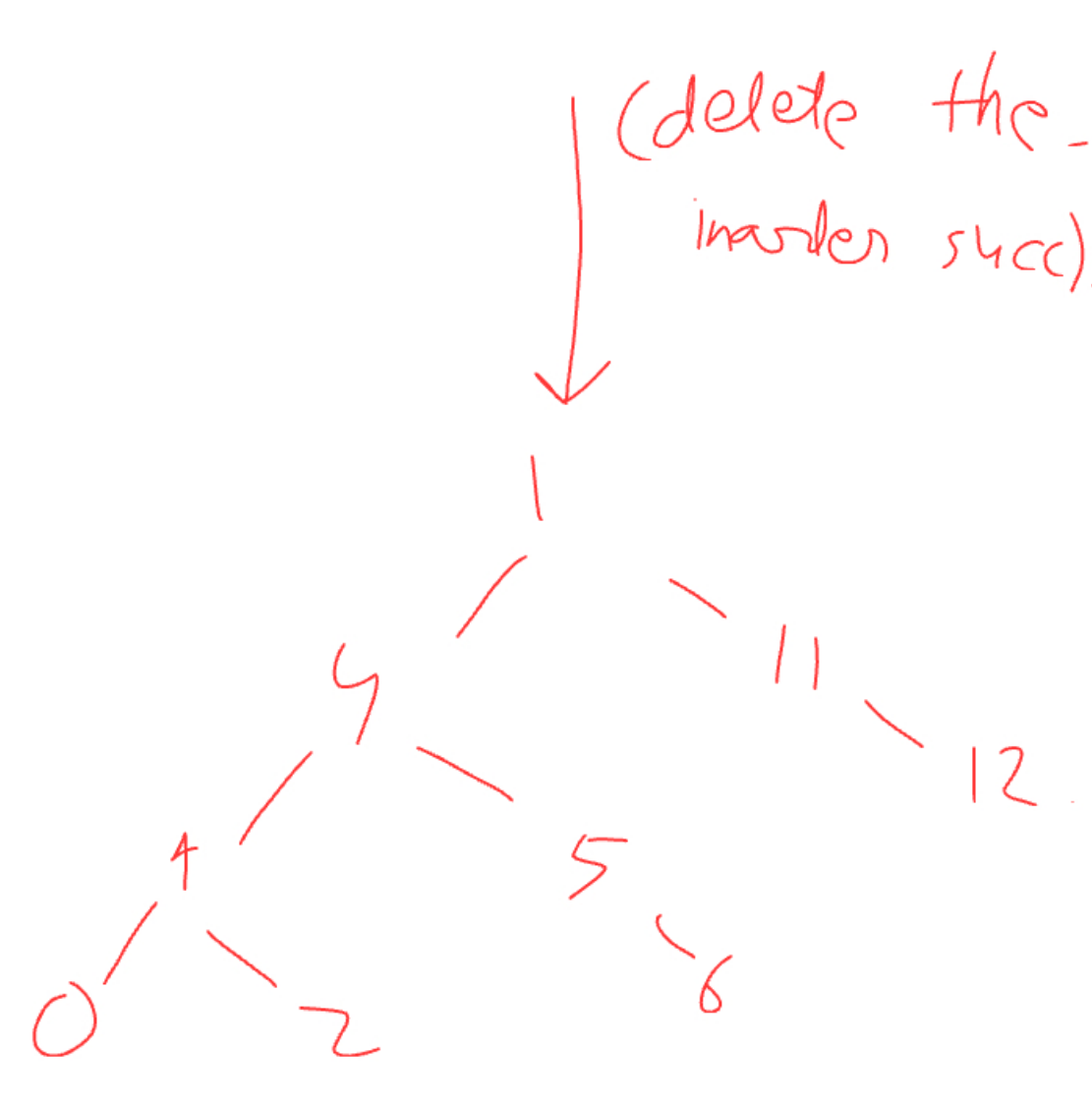
case-3 2 child nodes : (a) Replace the node with its inorder succ

(b) Delete the inorder succ

(Replaced 3 with its inorder succ)

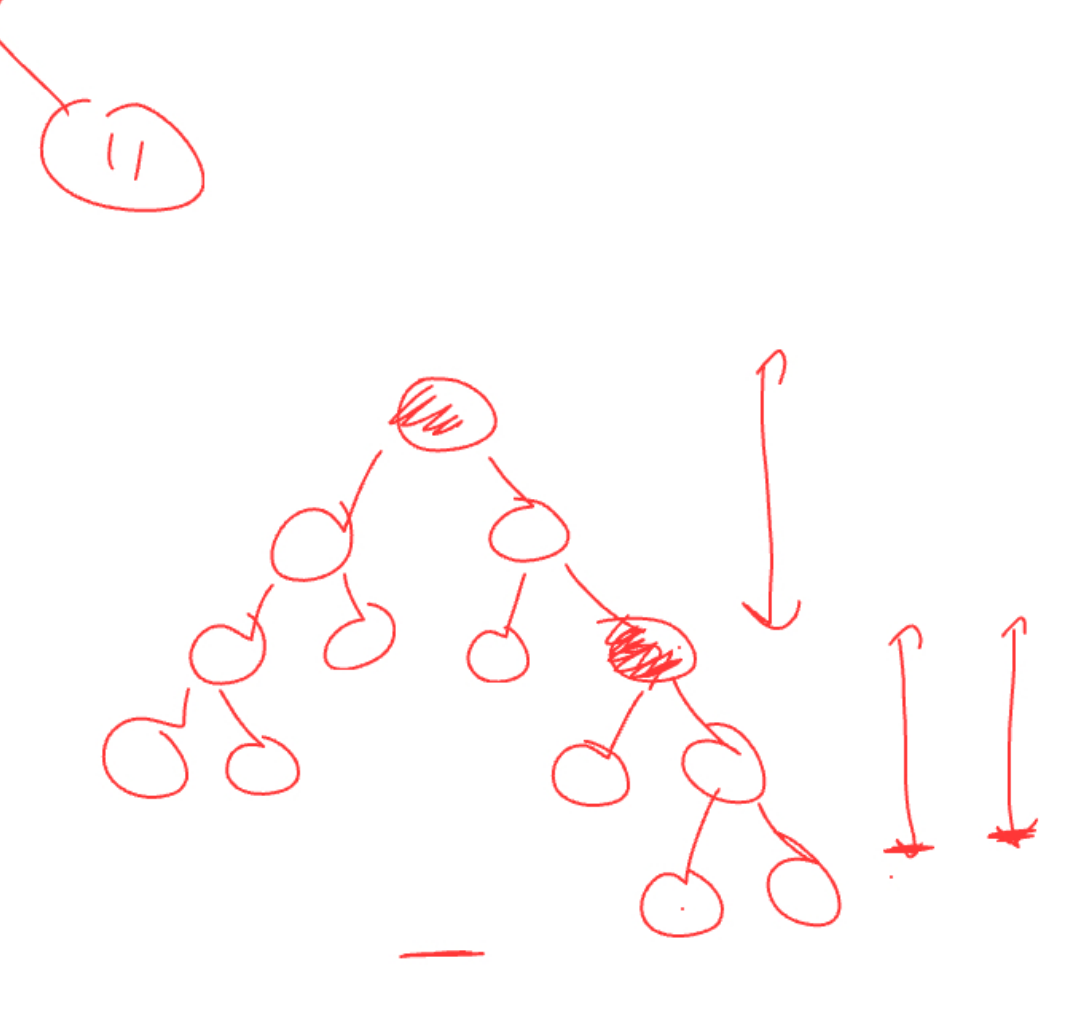


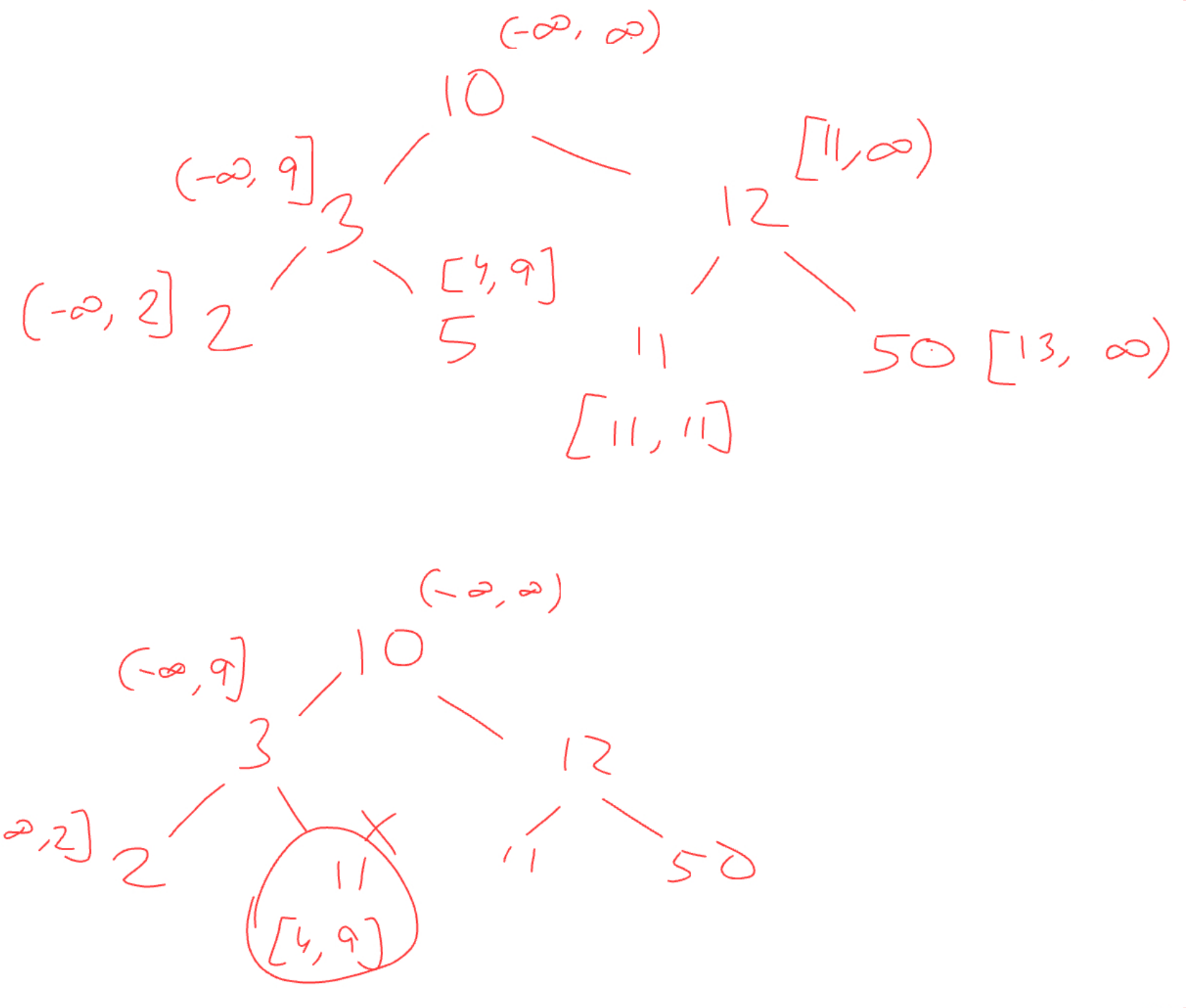
[0, 1, 2, 3, 4, 5, 6, 10, 11, 12]


(delete the inorder succ)

I/P : ↳ ⑩ value = 10 O/P : NULL

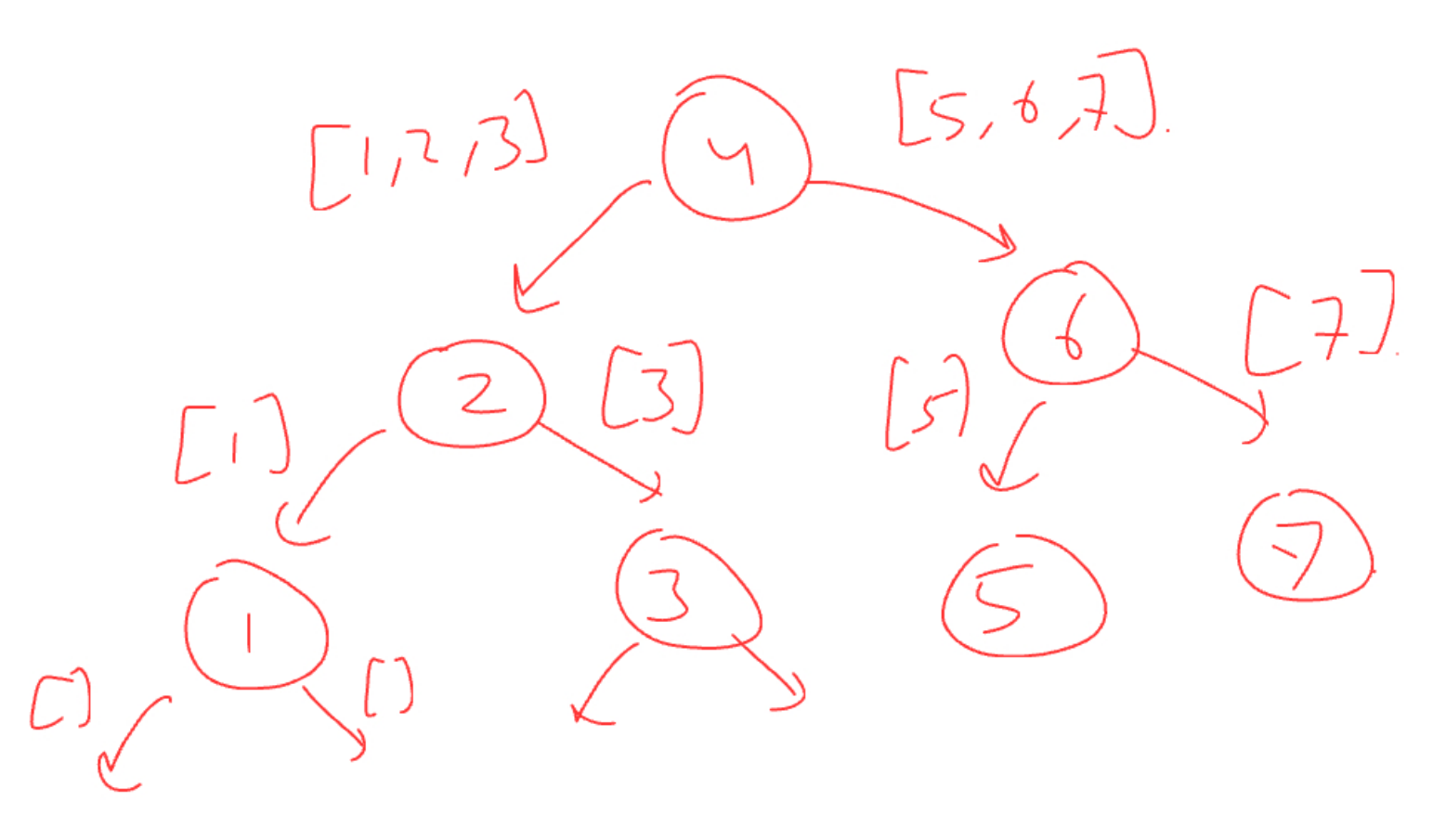
I/P : ↳ ⑩ — ⑪ value = 10 O/P : ⑪



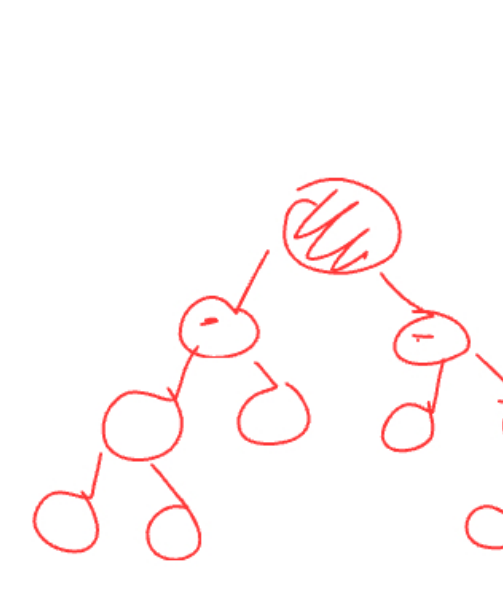

(-∞, ∞)
10
(-∞, 9]
3
[11, ∞)
12
(-∞, 2] 2
[4, 9]
5
11
[11, 11]
50 [13, ∞)
(-∞, ∞)
10
(-∞, 9]
3
(-∞, 2] 2
11
[4, 9]
12
11
50

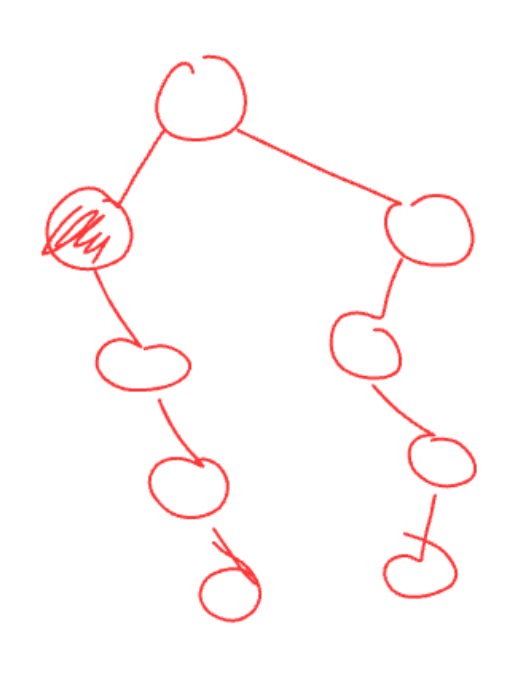
↓

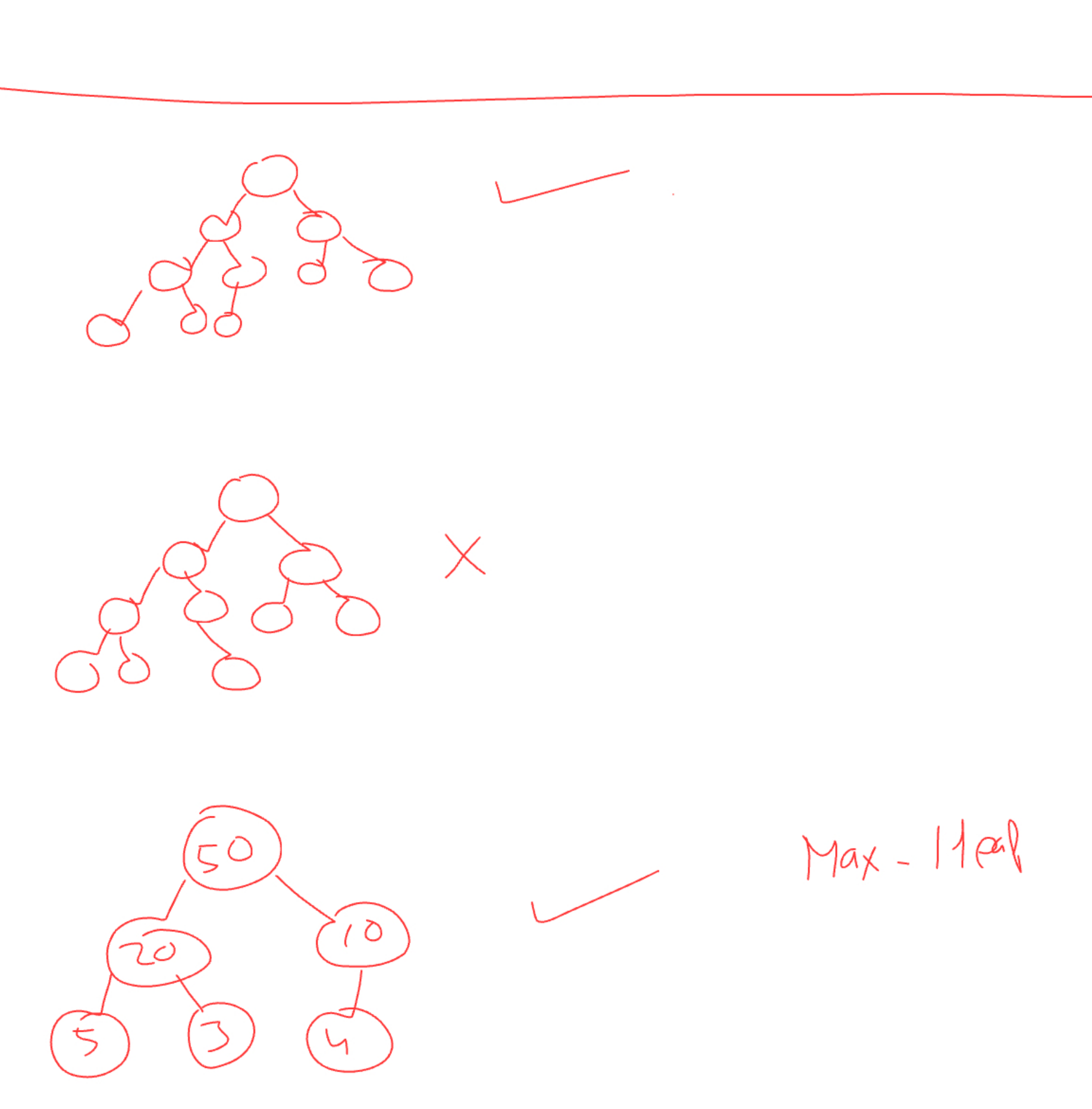
[1, 2, 3, 4, 5, 6, 7]

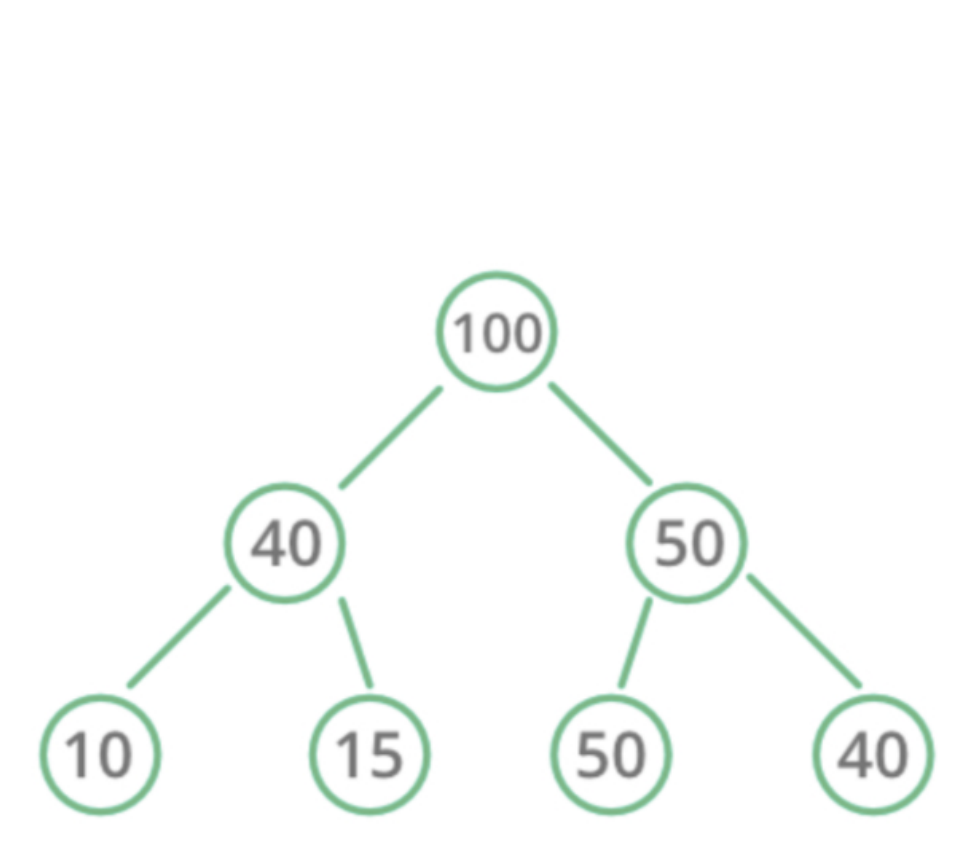

[1, 2, 3]
4
[5, 6, 7]
[1]
2
[3]
[5]
6
[7]
1
3
5
7
[]
[]

abs (lh - rh) ≤ 1





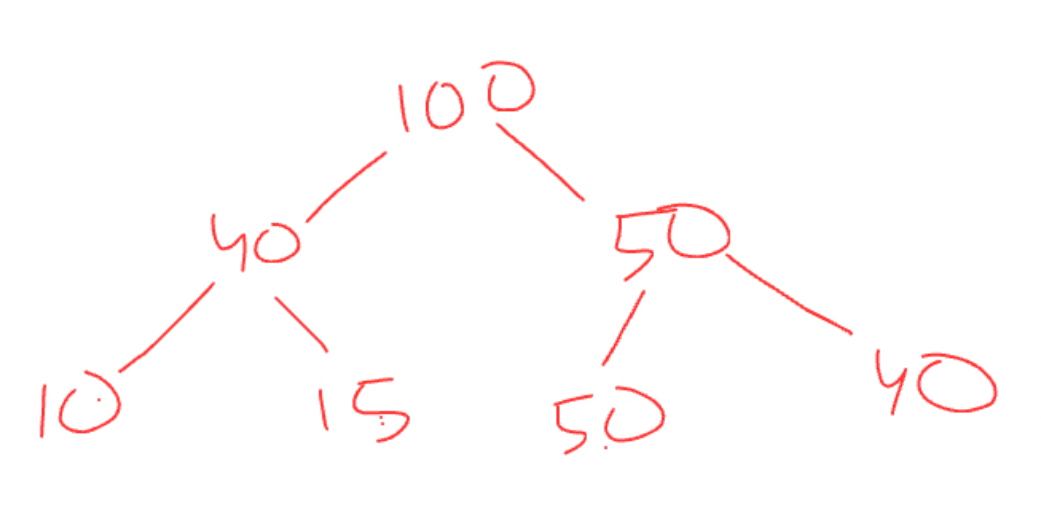

✓
X
50
20
10
5
3
4
✓
Max - Heap


100
40
50
10
15
50
40

100
40
5
35
37

0 1 2 3 4 5 6

100, 40, 50, 10, 15, 50, 40


100
40
50
10
15
50
40

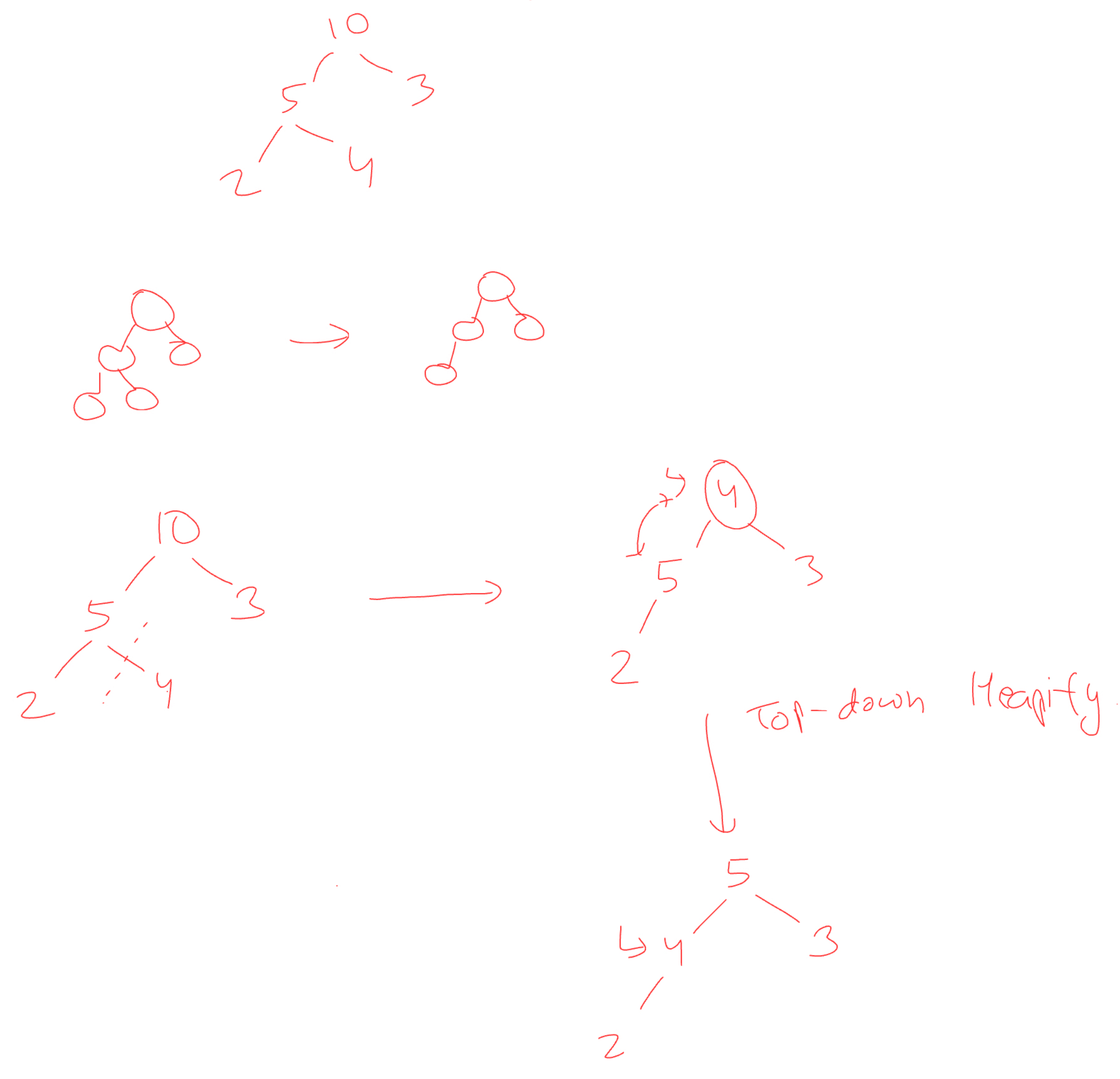
For node at index i :

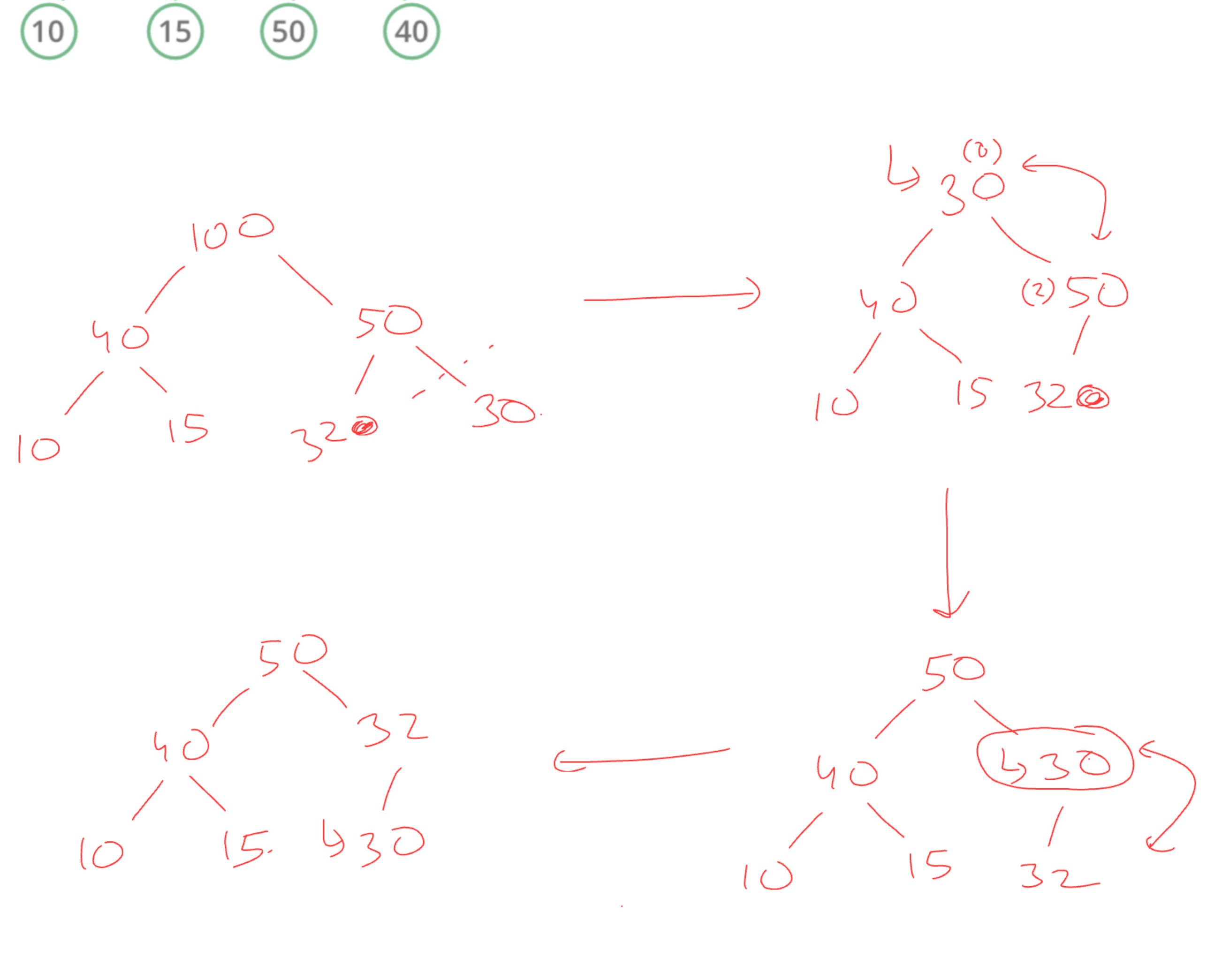
left child : (2 * i + 1)

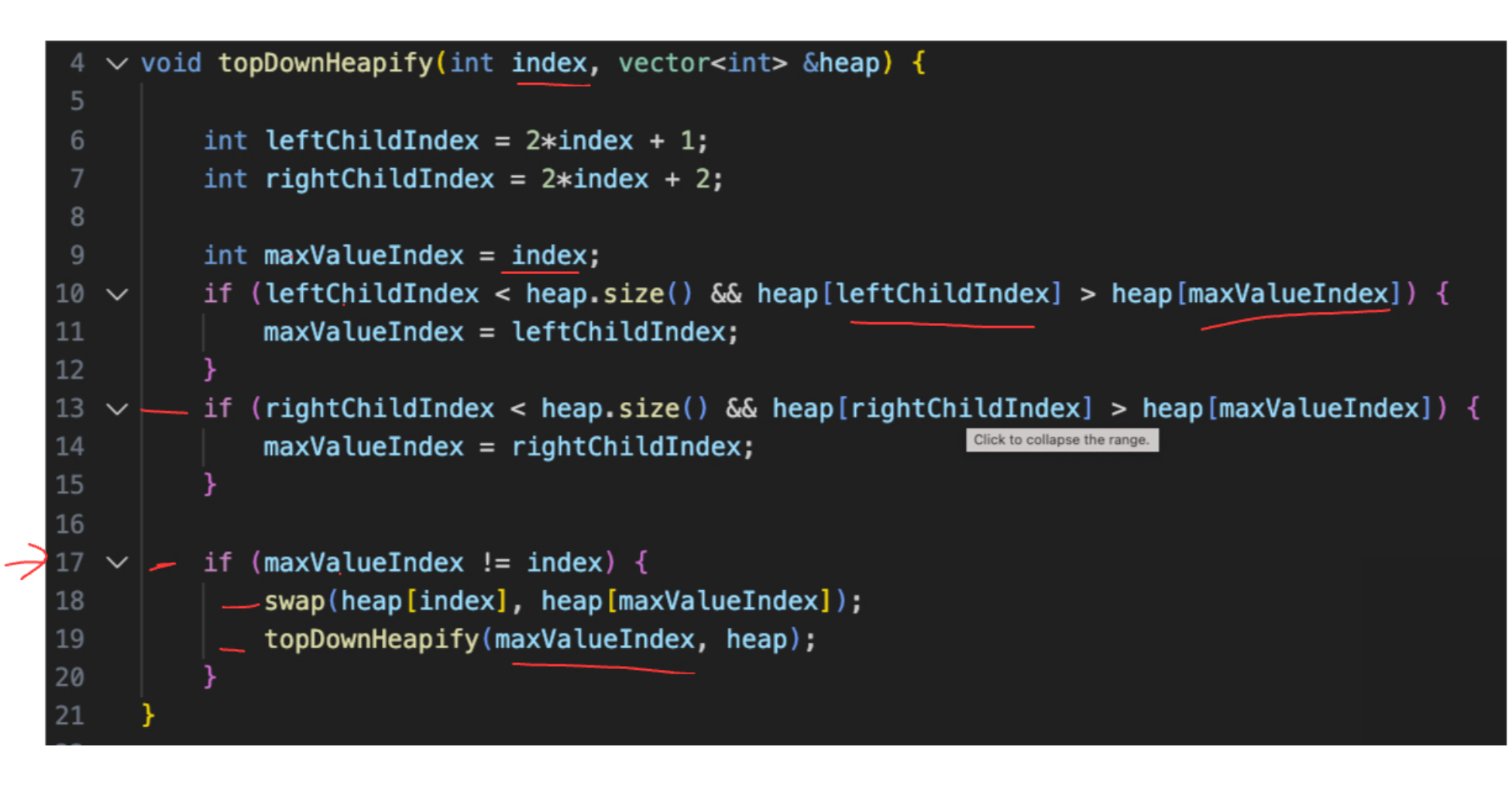
right child : (2 * i + 2)

parent : $\left(\frac{i-1}{2}\right)$

Delete.


10
5
3
2
4
10
5
3
2
4
4
5
3
2
Top-down Heapify
5
4
3
2


100
40
50
10
15
50
40
30
40
50
10
15
32
100
40
50
10
15
32
30
50
40
32
10
15
30
50
40
30
10
15
32



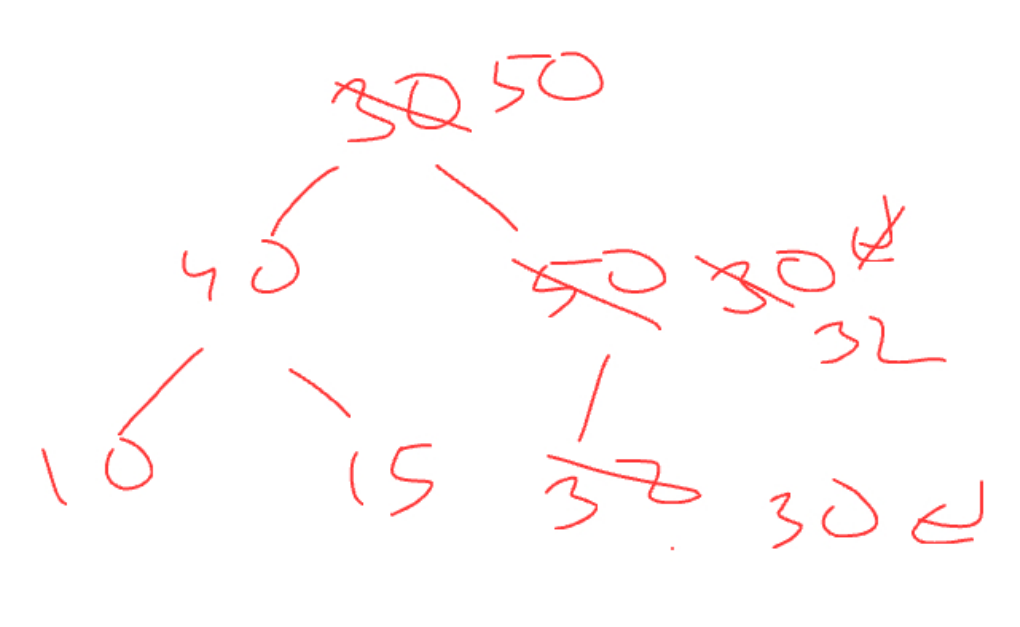
```
void topDownHeapify(int index, vector<int> &heap) {

    int leftChildIndex = 2*index + 1;
    int rightChildIndex = 2*index + 2;

    int maxValueIndex = index;
    if (leftChildIndex < heap.size() && heap[leftChildIndex] > heap[maxValueIndex]) {
        maxValueIndex = leftChildIndex;
    }
    if (rightChildIndex < heap.size() && heap[rightChildIndex] > heap[maxValueIndex]) {
        maxValueIndex = rightChildIndex;
    }

    if (maxValueIndex != index) {
        swap(heap[index], heap[maxValueIndex]);
        topDownHeapify(maxValueIndex, heap);
    }
}
```

0 1 2 3 4 5

[~~30~~, 40, ~~50~~, 10, 15, ~~32~~].

50 ~~30~~ 30

32


50
40
32
10
15
30

i = 0

lci = 1

rci = 2

mvi = ~~0~~ X 2

i = 5

lci = 11

rci = 12

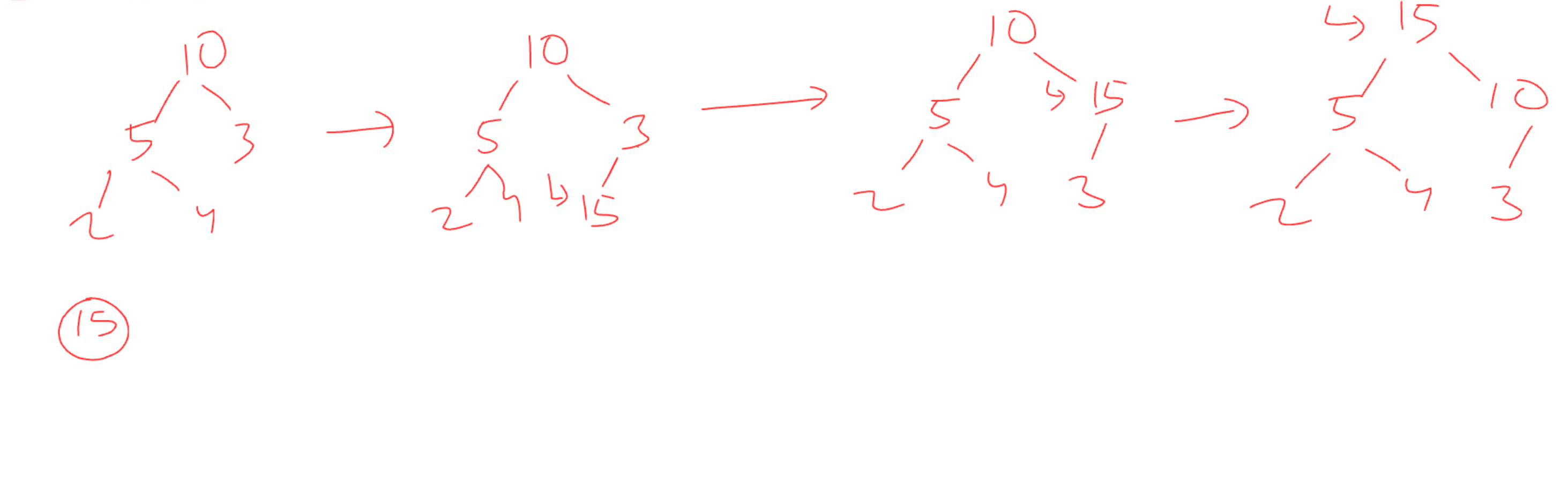
mvi = 5

i = 2

lci = 5

rci = 6

mvi = ~~2~~ 5


10
5
3
2
4
10
5
3
2
4
15
10
5
15
2
4
3
15
5
10
2
4
3

⑮